

Lab 03 - Embeddings and similarity

AI Agents for Business | TGS-2023018987 | v1.0

Objective

Represent tokens with a transformer and compare cosine with Euclidean similarity.

Alignment: K2, K4, K8, K10, K11; A2, A3. Scheduled hands-on time: 40 minutes, with trainer-led interpretation alongside the slides.

Files and prerequisites

- vectors.csv
- tickets.csv
- vocab.txt
- requirements.txt

Use Python 3.10 or newer and an organisation-approved LLM chat interface. The core local run requires no API key. Model outputs vary: assess evidence and behaviour, not matching prose. All business records are synthetic.

Detailed procedure

1. Create and activate a local virtual environment, then install requirements.txt as in Lab 2. All files required for this lab are included here.
2. Run python run.py. The script creates a small randomly initialised Hugging Face BERT encoder with 32 hidden dimensions, two layers and four attention heads.
3. Inspect output.json. Confirm each input token has a 32-value contextual representation and masked mean pooling produces one vector per sentence.
4. Calculate cosine and Euclidean values for $q=[1,0]$, $d1=[2,0]$, $d2=[1,1]$ using vectors.csv. Explain why cosine separates direction while Euclidean ties these two examples.
5. Compare the actual randomly initialised model similarities with the geometric example. State clearly that random model outputs demonstrate tensor mechanics, not reliable semantic ranking.
6. Draw an encoder block with embeddings, attention, feed-forward layers and pooling. Use PROMPTS.md to critique that explanation, then save the corrected diagram description and numerical results.

Acceptance checks

- The real encoder output shape is recorded.
- $\text{Cosine}(q,d1)=1$ and $\text{cosine}(q,d2)$ is about 0.7071.
- The report distinguishes encoder representation from autoregressive text generation.

Run python verify.py after run.py (use python3 if that is your interpreter command). The script verifies deterministic mechanics. Separately complete the evidence-based review of your own LLM output; no script claims an LLM task passed unless its output was actually inspected.

Troubleshooting

- Missing package: confirm the virtual environment is active, then install this folder's requirements.txt.

- File not found: open a terminal in this exact lab folder; the script resolves input paths relative to itself.
- Invalid JSON: remove Markdown fences and save a single JSON value; keep the original response for diagnosis.
- Unreliable model answer: reduce the task, include source IDs, inspect each claim and escalate missing evidence.
- Training results differ: record Python/package versions and seed; compare trends and limitations rather than claiming identical scores.

Evidence and cleanup

Keep output.json, learner-output.json and relevant screenshots or logs in your learner submission folder. Stop after verification; do not connect the exercise to real payment, email, HR or production systems. Local generated outputs can be archived after assessment; keep source data and prompts unchanged.